

Compte-rendu

Devoir N° 2 M202 Kotlin

Prénom : Aymen
Nom : EL YAAKOUBI
Groupe : DEVOAM202

Année scolaire : 2025/2026

Introduction.....	3
Exercice 1 :	4
Partie 1 : Prise de Commandes.....	4
Figure 1 : Classe Commande.....	4
Figure 2 : Classe Serveur.....	4
Partie 2 : Préparation des Plats.....	5
Figure 3 : Classe Cuisinier.....	5
Figure 4 : Classe Cuisinier.....	5
Partie 3 : Paiements.....	5
Figure 5 : Classe Caisse.....	6
Partie 4 : Gestion des Commandes avec Coroutines.....	6
Figure 6_1 : Classe Restaurant.....	7
Figure 6_2 : Classe Restaurant.....	8
Figure 7 : Classe Fun main().....	9
Figure 8 : Execution de L'exercice 1.....	10
Exercice 2 :	10
Partie 1 : Produits et Stock.....	10
Figure 1 : Classe StockException et data classe EvenementStock.....	10
Figure 2 : Classe Produit.....	11
Figure 3_1 : Classe Stock.....	11
Figure 3_2 : Classe Stock.....	12
Figure 4 : Classe Commande2.....	12
Partie 2 : Commandes Clients.....	12
Figure 5 : Classe GestionnaireCommandes.....	13
Partie 3 : Opérations Asynchrones sur le Stock.....	13
Figure 6 : Classe Entrepot.....	14
Figure 7_1 : Fun main().....	15
Figure 7_2 : Fun main().....	16
Figure 8_1 : Execution de L'exercice 2.....	16
Figure 8_2 : Execution de L'exercice 2.....	17
Figure 8_3 : Execution de L'exercice 2.....	17
Conclusion.....	18

Introduction

Le Devoir N°2 du module Kotlin (M202) nous demande de découvrir la programmation asynchrone avec les coroutines à travers deux projets pratiques. Le premier est une application de restaurant où une commande passe par trois étapes : le serveur la prend, les cuisiniers préparent les plats en même temps, puis la caisse s'occupe du paiement. Le deuxième est une application d'entrepôt qui gère un stock de produits : on peut ajouter ou retirer des quantités, traiter plusieurs commandes clients à la fois, lancer un inventaire et suivre toutes les modifications grâce à un Flow. L'idée de ce devoir est de montrer qu'un programme peut faire plusieurs choses à la fois sans jamais rester bloqué à attendre.

Exercice 1 :

Créer un système de gestion de commandes dans un restaurant utilisant les coroutines de Kotlin pour traiter les commandes, la préparation des plats, et la gestion des paiements de manière asynchrone.

Partie 1 : Prise de Commandes

```

package DevoirN1

import kotlinx.coroutines.*
import kotlin.coroutines.coroutineContext
import kotlin.random.Random

var prochainId = 1 1 Usage

class Commande1(val id: Int, val plats: List<String>, val total: Double) { 8 Usages
    fun afficherDetails() { 3 Usages
        println("Commande #${id} | Plats: ${plats.joinToString(separator = ", ")} | Total: $total DH")
    }
}
    
```

Figure 1 : Classe Commande

Classe Commande1 Elle représente une commande. Elle contient un id unique, la liste des plats commandés et le total à payer. Sa méthode afficherDetails() affiche ces informations.

```

class Serveur(val nom: String) { 4 Usages
    fun prendreCommande(plats: List<String>, prix: List<Double>): Commande1 { 1 Usage
        val commande = Commande1(id = prochainId++, plats, total = prix.sum())
        println("Serveur $nom a pris la commande #${commande.id}")
        return commande
    }

    fun afficherCommande(commandes: List<Commande1>) {
        println("Commandes de $nom :")
        for (commande in commandes) {
            commande.afficherDetails()
        }
    }
}
    
```

Figure 2 : Classe Serveur

Classe Serveur Elle représente un serveur, qui a un nom. La méthode prendreCommande(plats, prix) crée une nouvelle Commande1 avec un id unique et calcule le total en additionnant les prix. La méthode afficherCommande(commandes) affiche une liste de commandes.

Partie 2 : Préparation des Plats

```

class Cuisinier(val nom: String) { 4 Usages
    suspend fun preparerPlat(plat: String): String { 2 Usages
        println("$nom prépare $plat...")
        delay( timeMillis = 1000)
        return "$plat prêt (par $nom)"
    }

    suspend fun preparerCommande(commande: Commande1): List<String> = coroutineScope {
        commande.plats.map { plat -> async { preparerPlat(plat) } }.awaitAll()
    }
}
  
```

Figure 3 : Classe Cuisinier

Classe Cuisinier Elle représente un cuisinier, qui a un nom. La méthode `preparerPlat(plat)` simule la préparation d'un plat avec une pause d'une seconde (`delay`). La méthode `preparerCommande(commande1)` lance la préparation de tous les plats en même temps avec `async`, puis attend tous les résultats avec `awaitAll()`.

```

class Cuisine(val cuisiniers: List<Cuisinier>) { 2 Usages
    suspend fun gererPreparationCommande(commande: Commande1): List<String> = coroutineScope {
        commande.plats.mapIndexed { index, plat ->
            val cuisinier = cuisiniers[index % cuisiniers.size]
            async { cuisinier.preparerPlat(plat) }
        }.awaitAll()
    }
}
  
```

Figure 4 : Classe Cuisine

Classe Cuisine Elle contient la liste des cuisiniers. La méthode `gererPreparationCommande(commande1)` distribue les plats entre les cuisiniers, chacun son tour. Tous les plats sont préparés en même temps grâce à `async`, et la méthode attend que tout soit prêt.

Partie 3 : Paiements

```

class Caisse { 2 Usages
    private val paiements = mutableMapOf<Int, Job>() 4 Usages

    suspend fun traiterPaielement(commande: Commande1): Boolean { 1 Usage
        paiements[commande.id] = coroutineContext[Job]!!
        println("Paielement de la commande #${commande.id} en cours...")
        delay( timeMillis = 1500)
        if (Random.nextInt( until = 100) < 30) {
            throw Exception("Paielement refusé pour la commande #${commande.id}")
        }
        paiements.remove( key = commande.id)
        return true
    }

    fun annulerPaielement(commande: Commande1) { 1 Usage
        paiements[commande.id]?.cancel()
        paiements.remove( key = commande.id)
    }
}
  
```

Figure 5 : Classe Caisse

Classe Caisse Elle gère les paiements. La méthode `traiterPaielement(commande1)` simule un paiement asynchrone d'une seconde et demie, avec environ 30 % de chances d'échec : dans ce cas, elle lance une exception. Elle garde aussi la coroutine du paiement (son Job). La méthode `annulerPaielement(commande1)` utilise ce Job pour annuler le paiement en cours avec `cancel()`.

Partie 4 : Gestion des Commandes avec Coroutines

```

class Restaurant( 1 Usage
    val serveurs: List<Serveur>,
    val cuisine: Cuisine,
    val caisse: Caisse
) {
    val commandes = mutableListOf<Commande1>() 4 Usages

    suspend fun prendreCommandeEtTraiter( 3 Usages
        serveur: Serveur, plats: List<String>, prix: List<Double>) {
        val commande = serveur.prendreCommande(plats, prix)
        commandes.add(commande)
        commande.afficherDetails()
        try {
            val resultat = cuisine.gererPreparationCommande(commande)
            println("Commande #${commande.id} prête : $resultat")
            if (caisse.traiterPaieement(commande)) {
                println("Commande #${commande.id} payée avec succès")
            }
        } catch (e: CancellationException) {
            println("Commande #${commande.id} annulée")
            throw e
        } catch (e: Exception) {
            println("Erreur : ${e.message}")
        } finally {
            commandes.remove(element = commande)
        }
    }
}
    
```

Figure 6_1 : Classe Restaurant

Classe Restaurant C'est la classe principale qui relie tout. Elle contient la liste des commandes en cours, les serveurs, la cuisine et la caisse. La méthode prendreCommandeEtTraiter(...) enchaîne les trois étapes : prise de commande, préparation, puis paiement. Les erreurs de paiement sont gérées avec try / catch, et la commande est retirée de la liste à la fin avec finally.

```

fun afficherCommandesEnCours() { 1 Usage
    println("--- Commandes en cours ---")
    for (commande in commandes) {
        commande.afficherDetails()
    }
}

fun annulerCommande(id: Int) { 1 Usage
    val commande = commandes.find { it.id == id }
    if (commande != null) {
        caisse.annulerPaieement(commande)
    }
}
}
    
```

Figure 6_2 : Classe Restaurant

La méthode `afficherCommandesEnCours()` affiche les commandes en cours. J'ai ajouté la méthode `annulerCommande(id)` pour pouvoir annuler le paiement d'une commande à partir de son numéro.

```

fun main() = runBlocking<Unit> {
    val cuisine = Cuisine( cuisiniers = listOf(Cuisinier( nom = "Ali"),
        Cuisinier( nom = "Sara"), Cuisinier( nom = "Omar")))
    val restaurant = Restaurant( serveurs = listOf(Serveur( nom = "Youssef"),
        Serveur( nom = "Imane")), cuisine, Caisse())
    launch {
        restaurant.prendreCommandeEtTraiter(
            serveur = restaurant.serveurs[0],
            plats = listOf("Pizza", "Salade"),
            prix = listOf(60.0, 25.0)
        ) }
    launch {
        restaurant.prendreCommandeEtTraiter(
            serveur = restaurant.serveurs[1],
            plats = listOf("Tajine", "Couscous", "Soupe"),
            prix = listOf(70.0, 65.0, 20.0)
        ) }
}
    
```

```

launch {
    restaurant.prendreCommandeEtTraiter(
        serveur = restaurant.serveurs[0],
        plats = listOf("Burger"),
        prix = listOf(45.0)
    ) }
delay( timeMillis = 500)
restaurant.afficherCommandesEnCours()
delay( timeMillis = 1300)
restaurant.annulerCommande( id = 2) }
    
```

Figure 7 : Classe Fun main()

Fonction main Elle crée la cuisine avec trois cuisiniers et le restaurant avec deux serveurs. Elle lance trois commandes en parallèle avec launch, affiche les commandes en cours, puis annule la commande numéro 2 pour montrer la gestion de l'annulation.

```

Serveur Youssef a pris la commande #1
Commande #1 | Plats: Pizza, Salade | Total: 85.0 DH
Serveur Imane a pris la commande #2
Commande #2 | Plats: Tajine, Couscous, Soupe | Total: 155.0 DH
Serveur Youssef a pris la commande #3
Commande #3 | Plats: Burger | Total: 45.0 DH
Ali prépare Pizza...
Sara prépare Salade...
Ali prépare Tajine...
Sara prépare Couscous...
Omar prépare Soupe...
Ali prépare Burger...
--- Commandes en cours ---
Commande #1 | Plats: Pizza, Salade | Total: 85.0 DH
Commande #2 | Plats: Tajine, Couscous, Soupe | Total: 155.0 DH
Commande #3 | Plats: Burger | Total: 45.0 DH
Commande #1 prête : [Pizza prêt (par Ali), Salade prêt (par Sara)]
Paiement de la commande #1 en cours...
Commande #2 prête : [Tajine prêt (par Ali), Couscous prêt (par Sara), Soupe prêt (par Omar)]
Paiement de la commande #2 en cours...
Commande #3 prête : [Burger prêt (par Ali)]
Paiement de la commande #3 en cours...
Commande #2 annulée
Erreur : Paiement refusé pour la commande #1
Commande #3 payée avec succès
    
```

Figure 8 : Execution de L'exercice 1

Le programme lance trois commandes en même temps : les serveurs les prennent, puis les cuisiniers (Ali, Sara et Omar) préparent tous les plats en parallèle pendant une seconde. Une fois chaque commande prête, elle passe à la caisse pour le paiement, qui dure une seconde et demie. Pendant ce temps, main affiche les commandes en cours, puis annule la commande #2, qui ne sera donc jamais payée. Le paiement de la commande #1 échoue au hasard et l'erreur est affichée, alors que la commande #3 est payée avec succès, et ce résultat peut changer à chaque exécution.

Exercice 2 :

Créer un système de gestion de stock pour un entrepôt en utilisant les coroutines Kotlin. Ce système doit gérer l'ajout et le retrait de produits de manière asynchrone, tout en offrant la possibilité d'exécuter des inventaires et de traiter des commandes clients en parallèle. Les différentes opérations sur le stock doivent être réalisées de manière non bloquante afin d'optimiser l'utilisation des ressources.

Partie 1 : Produits et Stock

```

package DevoirN1

import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock

class StockException(message: String) : Exception(message) 5 Usages

data class EvenementStock(val idProduit: Int, val type: String, val quantite: Int) 4 Usages
  
```

Figure 1 : Classe StockException et data classe EvenementStock

Classe StockException (ajoutée) C'est une exception personnalisée. Elle est lancée quand un produit n'existe pas ou quand le stock est insuffisant. Classe EvenementStock (ajoutée) Elle représente un événement de mise à jour du stock. Elle contient l'id du produit, le type d'événement (AJOUT_PRODUIIT, AJOUT ou RETRAIT) et la quantité concernée.

```

class Produit(val id: Int, val nom: String, var quantite: Int) { 6 Usages
    fun afficherDetails() { 1 Usage
        println("Produit #${id} | $nom | Quantité: $quantite")
    }
}
  
```

Figure 2 : Classe Produit

Classe Produit Elle représente un produit avec un id, un nom et une quantite en stock. La méthode `afficherDetails()` affiche ces informations.

```

class Stock { 2 Usages
    val produits = mutableMapOf<Int, Produit>() 5 Usages
    val evenements = MutableSharedFlow<EvenementStock>(extraBufferCapacity = 64) 4 Usages
    private val mutex = Mutex() 3 Usages

    fun ajouterProduit(produit: Produit) { 1 Usage
        produits[produit.id] = produit
        evenements.tryEmit(value = EvenementStock(idProduit = produit.id, type = "AJOUT_PRODUIT", produit.quantite))
    }

    suspend fun ajouterQuantite(idProduit: Int, quantite: Int) { 1 Usage
        delay(timeMillis = 200)
        mutex.withLock {
            val produit = produits[idProduit] ?: throw StockException("Produit $idProduit introuvable")
            produit.quantite += quantite
        }
        evenements.emit(value = EvenementStock(idProduit, type = "AJOUT", quantite))
    }
}
  
```

Figure 3_1 : Classe Stock

Classe Stock Elle contient tous les produits dans une MutableMap (id → produit). La méthode `ajouterProduit` ajoute un nouveau produit. Les méthodes `ajouterQuantite` ...

```

suspend fun retirerQuantite(idProduit: Int, quantite: Int) { 2 Usages
    delay(timeMillis = 200)
    mutex.withLock {
        val produit = produits[idProduit] ?: throw StockException("Produit $idProduit introuvable")
        if (produit.quantite < quantite) {
            throw StockException(
                "Stock insuffisant pour ${produit.nom} (disponible: ${produit.quantite}, demandé: $quantite)"
            )
        }
        produit.quantite -= quantite
        evenements.emit(value = EvenementStock(idProduit, type = "RETRAIT", quantite))
    }
}

suspend fun estDisponible(idProduit: Int, quantite: Int): Boolean { 1 Usage
    return mutex.withLock {
        (produits[idProduit]?.quantite ?: 0) >= quantite
    }
}

fun afficherStock() { 2 Usages
    println("--- Stock actuel ---")
    for (produit in produits.values) {
        produit.afficherDetails()
    }
}
  
```

Figure 3_2 : Classe Stock

... et retirerQuantite modifie la quantité de manière asynchrone ; retirerQuantite lance une exception si la quantité demandée est trop grande. Un Mutex empêche deux coroutines de modifier le stock en même temps. Chaque modification envoie un événement dans un MutableSharedFlow. La méthode afficherStock() affiche tous les produits.

```

class Commande2(val idCommande: Int, val produits: List<Pair<Int, Int>>) { 6 Usages
    fun afficherCommande() { 1 Usage
        println("Commande #${idCommande}")
        for ((idProduit, quantite) in produits) {
            println("  Produit ID: $idProduit | Quantité demandée: $quantite")
        }
    }
}
    
```

Figure 4 : Classe Commande2

Classe Commande Elle représente une commande client avec un idCommande et une liste de paires (id du produit, quantité commandée). La méthode afficherCommande() affiche ces informations.

Partie 2 : Commandes Clients

```

class GestionnaireCommandes(val stock: Stock) { 1 Usage
    val commandes = mutableListOf<Commande2>() 1 Usage

    suspend fun traiterCommande(commande: Commande2) { 1 Usage
        commandes.add(commande)
        commande.afficherCommande()
        for ((idProduit, quantite) in commande.produits) {
            if (!stock.estDisponible(idProduit, quantite)) {
                println("Commande #${commande.idCommande} refusée : produit " +
                    "$idProduit indisponible (quantité $quantite)")
                return }
            try {
                for ((idProduit, quantite) in commande.produits) {
                    stock.retirerQuantite(idProduit, quantite)
                }
                println("Commande #${commande.idCommande} traitée avec succès")
            } catch (e: StockException) {
                println("Commande #${commande.idCommande} refusée : ${e.message}")
            }
        }
    }

    suspend fun gererCommandes(commandes: List<Commande2>) = coroutineScope { 1 Usage
        for (commande in commandes) {
            launch { traiterCommande(commande) }
        }
    }
}
    
```

Figure 5 : Classe GestionnaireCommandes

Classe GestionnaireCommandes Elle traite les commandes à l'aide du stock. La méthode `traiterCommande(commande)` vérifie d'abord que tous les produits sont disponibles, puis retire les quantités du stock. Si un produit manque, la commande est refusée. La méthode `gererCommandes(commandes)` lance chaque commande dans sa propre coroutine avec `launch`, donc elles sont traitées en parallèle.

Partie 3 : Opérations Asynchrones sur le Stock

```

class Entrepot(val scope: CoroutineScope) { 1 Usage
    val stock = Stock() 5 Usages
    val gestionnaireCommandes = GestionnaireCommandes(stock) 1 Usage

    fun ajouterProduitAuStock(produit: Produit): Job { 3 Usages
        return scope.launch {
            stock.ajouterProduit(produit) } }

    fun retirerProduitDuStock(idProduit: Int, quantite: Int): Job { 2 Usages
        return scope.launch {
            try {
                stock.retirerQuantite(idProduit, quantite)
                println("Retrait de $quantite du produit $idProduit effectué")
            } catch (e: StockException) {
                println("Erreur : ${e.message}") } } }

    fun gererInventaire(): Job { 1 Usage
        return scope.launch {
            println("Inventaire en cours...")
            delay(timeMillis = 500)
            stock.afficherStock()
            println("Inventaire terminé") } } }
  
```

Figure 6 : Classe Entrepot

Classe Entrepot Elle regroupe le stock et le gestionnaireCommandes. La méthode `ajouterProduitAuStock` ajoute un produit avec `launch`. La méthode `retirerProduitDuStock` retire une quantité avec `launch` et affiche un message si le stock est insuffisant. La méthode `gererInventaire()` fait l'inventaire dans une coroutine séparée, pendant que d'autres tâches continuent.

```

fun main() = runBlocking<Unit> {
    val entrepot = Entrepot(scope = this)
    val stock = entrepot.stock
    val inventaire = mutableMapOf<Int, Int>()

    val collecteur = launch(start = CoroutineStart.UNDISPATCHED) {
        stock.events.collect { evenement ->
            val actuel = inventaire[eventement.idProduit] ?: 0
            when (evenement.type) {
                "AJOUT_PRODUIT" -> inventaire[eventement.idProduit] = evenement.quantite
                "AJOUT" -> inventaire[eventement.idProduit] = actuel + evenement.quantite
                "RETRAIT" -> inventaire[eventement.idProduit] = actuel - evenement.quantite
            }
            println("[Flow] ${evenement.type} produit ${evenement.idProduit} " +
                "x${evenement.quantite} -> $inventaire") } }

    println("=== 1. Ajout des produits ===")
    listOf(
        entrepot.ajouterProduitAuStock(Produit(id = 1, nom = "Laptop", quantite = 10)),
        entrepot.ajouterProduitAuStock(Produit(id = 2, nom = "Souris", quantite = 50)),
        entrepot.ajouterProduitAuStock(Produit(id = 3, nom = "Clavier", quantite = 30))
    ).joinAll()
  }

```

Figure 7_1 : Fun main()

runBlocking crée la zone où les coroutines s'exécutent et attend qu'elles finissent. On crée l'entrepôt et une table inventaire pour suivre les quantités en temps réel. Le collecteur écoute le Flow : à chaque événement du stock (AJOUT_PRODUIT, AJOUT ou RETRAIT), il met à jour inventaire et l'affiche. Ensuite, l'étape 1 ajoute trois produits (Laptop, Souris, Clavier) en parallèle, et joinAll() attend que les trois soient finis.

```

println("=== 3. Commandes en parallèle et inventaire ===")
val inventaireJob = entrepot.gererInventaire()
entrepot.gestionnaireCommandes.gererCommandes(
    listOf(
        Commande2(idCommande = 1, produits = listOf(1 to 2, 2 to 5)),
        Commande2(idCommande = 2, produits = listOf(3 to 10)),
        Commande2(idCommande = 3, produits = listOf(1 to 100))
    )
)
inventaireJob.join()

println("=== 4. Stock final ===")
stock.afficherStock()

delay(timeMillis = 300)
collecteur.cancel()
}

```

Figure 7_2 : Fun main()

L'inventaire démarre en arrière-plan pendant que trois commandes sont traitées en parallèle. La commande 3 sera refusée, car elle demande 100 Laptops et il n'y en a pas assez. Quand tout est terminé, on affiche le stock final. Le `delay(300)` laisse le temps au collecteur d'afficher les derniers événements, puis `cancel()` l'arrête, sinon le programme ne se terminerait jamais.

```

=== 1. Ajout des produits ===
[Flow] AJOUT_PRODUIT produit 1 x10 -> {1=10}
[Flow] AJOUT_PRODUIT produit 2 x50 -> {1=10, 2=50}
[Flow] AJOUT_PRODUIT produit 3 x30 -> {1=10, 2=50, 3=30}
=== 2. Ajout et retrait asynchrones ===
[Flow] AJOUT produit 1 x5 -> {1=15, 2=50, 3=30}
Retrait de 20 du produit 2 effectué
[Flow] RETRAIT produit 2 x20 -> {1=15, 2=30, 3=30}
Erreur : Stock insuffisant pour Clavier (disponible: 30, demandé: 100)
    
```

Figure 8_1 : Execution de L'exercice 2

Le programme ajoute trois produits (Laptop 10, Souris 50, Clavier 30), et chaque ajout envoie un événement [Flow] qui met à jour l'inventaire en temps réel. Ensuite, il ajoute 5 Laptops (1=15) et retire 20 Souris (2=30), mais le retrait de 100 Claviers échoue avec une erreur, car il n'y en a que 30.

```

=== 3. Commandes en parallèle et inventaire ===
Inventaire en cours...
Commande #1
    Produit ID: 1 | Quantité demandée: 2
    Produit ID: 2 | Quantité demandée: 5
Commande #2
    Produit ID: 3 | Quantité demandée: 10
Commande #3
    Produit ID: 1 | Quantité demandée: 100
Commande #3 refusée : produit 1 indisponible (quantité 100)
Commande #2 traitée avec succès
[Flow] RETRAIT produit 1 x2 -> {1=13, 2=30, 3=30}
[Flow] RETRAIT produit 3 x10 -> {1=13, 2=30, 3=20}
Commande #1 traitée avec succès
[Flow] RETRAIT produit 2 x5 -> {1=13, 2=25, 3=20}
    
```

Figure 8_2 : Execution de L'exercice 2

Les trois commandes sont traitées en même temps pendant que l'inventaire démarre en arrière-plan. La commande #3 est refusée, car elle demande 100 Laptops alors qu'il n'y en a que 15, alors que les commandes #1 et #2 réussissent. Chaque retrait envoie un événement [Flow] qui met à jour l'inventaire : 1=13, 3=20, puis 2=25.

```
--- Stock actuel ---  
Produit #1 | Laptop | Quantité: 13  
Produit #2 | Souris | Quantité: 25  
Produit #3 | Clavier | Quantité: 20  
Inventaire terminé  
=== 4. Stock final ===  
--- Stock actuel ---  
Produit #1 | Laptop | Quantité: 13  
Produit #2 | Souris | Quantité: 25  
Produit #3 | Clavier | Quantité: 20  
  
Process finished with exit code 0
```

Figure 8_3 : Execution de L'exercice 2

L'inventaire affiche le stock actuel puis « Inventaire terminé », et l'étape 4 affiche le stock final : Laptop 13, Souris 25 et Clavier 20. Le message `exit code 0` signifie que le programme s'est terminé sans erreur.

Conclusion

Ce devoir nous a permis de mettre en pratique les coroutines de Kotlin dans deux situations concrètes. Nous avons vu que les coroutines rendent le code asynchrone simple à écrire, presque aussi lisible que du code normal, tout en évitant de bloquer le programme. Les notions de `launch`, `async / await`, d'annulation, de gestion des exceptions, de `Mutex` et de `Flow` sont des bases importantes pour le développement mobile, où les applications doivent rester fluides pendant les opérations longues.

<https://github.com/Aymen5285356/Devoir-N2-Kotlin>